UNIVERSIDAD AUTÓNOMA DE SINALOA

Facultad de Informática Culiacán
Licenciatura en Informática

Análisis y Diseño Orientado a Objetos

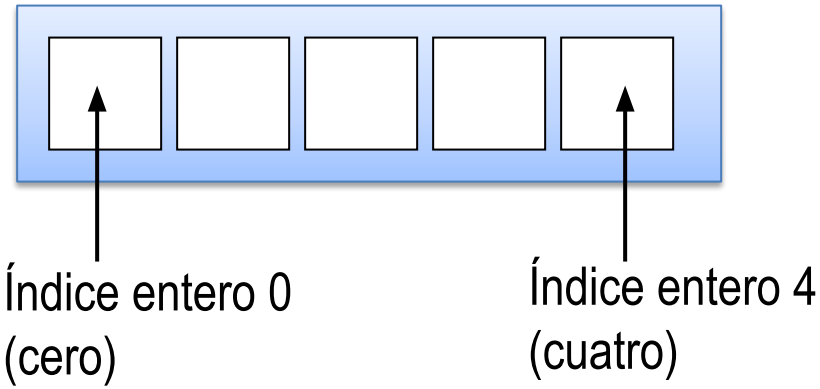
Fundamentos de Programación

M.C. Gerardo Beltrán

Arreglos

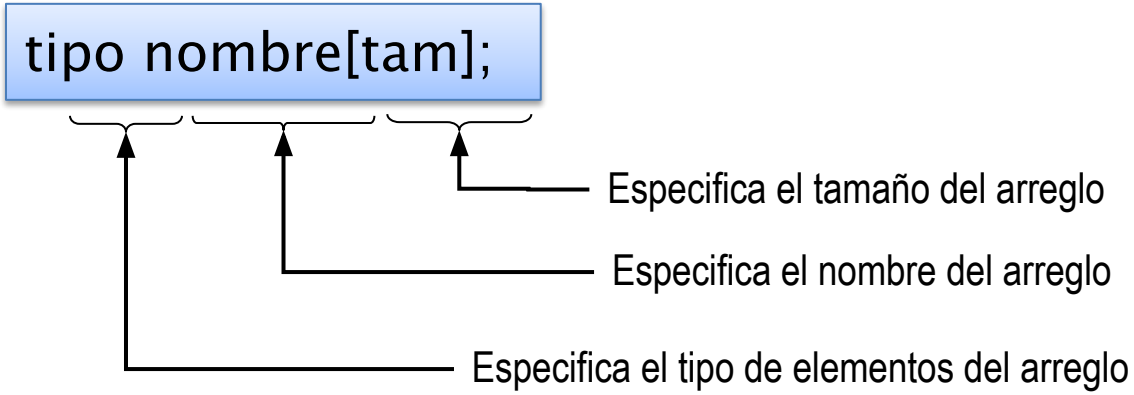
Arreglos

- un arreglo es una secuencia de elementos
  - Todos los elementos de un arreglo son del mismo tipo
  - Se accede a elementos individuales usando índices enteros



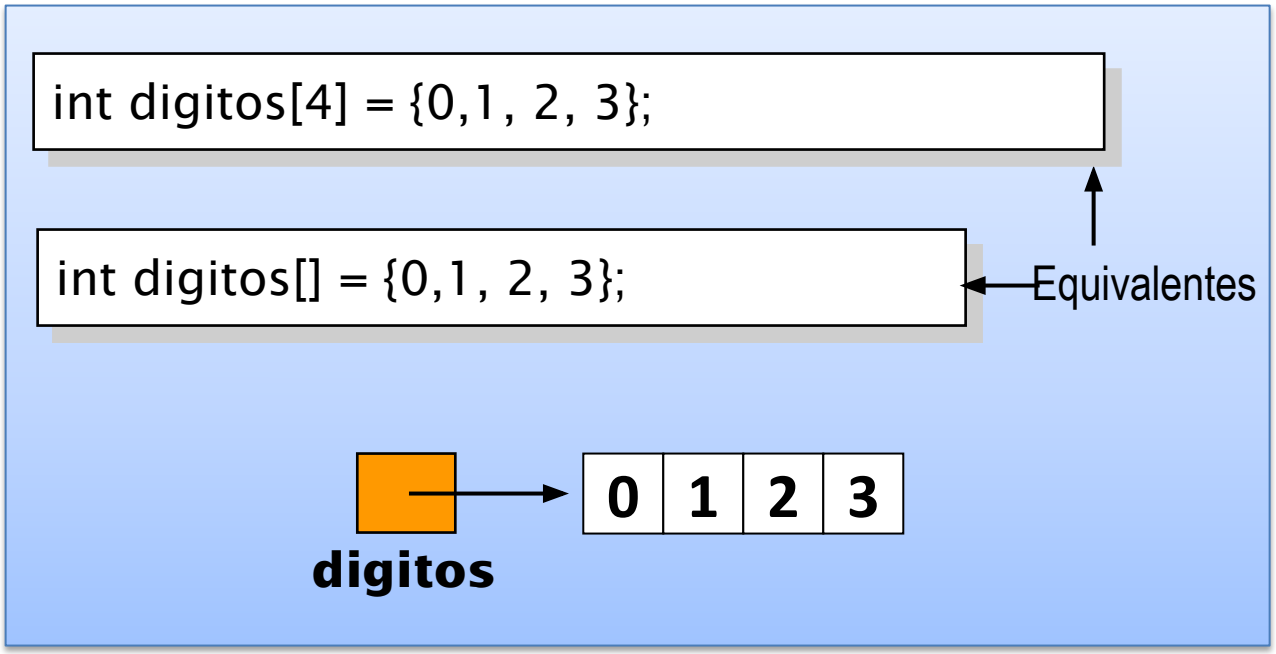
Declaración de arreglos

- Una variable arreglo se declara especificando:
  - El tipo de elementos del arreglo
  - El nombre de la variable
  - El tamaño del arreglo



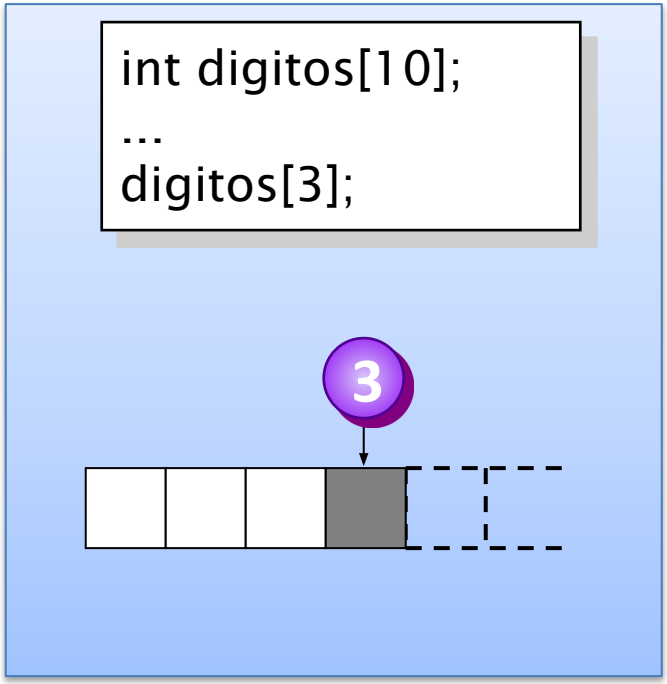
Inicialización un arreglo

- Es posible inicializar explícitamente los elementos de un arreglo
  - Se puede utilizar una expresión abreviada



Acceso a los elementos de un vector

- Se indica un índice entero para el acceso
  - Los índices se cuentan a partir de cero
  - Es responsabilidad del programador el control del tamaño del arreglo



Acceso a los elementos de un arreglo

- Acceso secuencial a los elementos de un arreglo

```
int digitos[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8 ,9};
for ( i = 0; i < 10; i++) {
  printf("El valor de elemento %d es %d\n", i,  digitos[i]);
}
```

0 1 2 3 4 5 6 7 8 9

Arreglos y Funciones

- En el caso del paso de un arreglo como parámetro a una función, es importante enviar además el tamaño del arreglo (Arreglos numéricos o de estructuras).

- El arreglo enviado como parámetro a la función es pasado por referencia, es decir se pasa la dirección del primer elemento y mediante el índice se tiene acceso a todos los elementos.

- Por lo anterior el arreglo puede ser modificado e indirectamente modificar la variable de origen de los datos.

Arreglos y Funciones - Ejemplo

```
#include <stdio.h>

int sumatoria (int vector[], int N);
void main (){
  int resultado, datos[]={19,20,36,45,54,69,70,88,97,101};
  resultado = sumatoria( datos, 10 );
  printf("La suma es: %d\n", resultado);
}

int sumatoria(int vector[], int N){
  int i, suma = 0;
   for (i=0; i< N; i++)
       suma += vector[i];
  return suma;
}
```

Arreglos y Funciones - Ejemplo

```
#include <stdio.h>
void incrementar (int vector[], int N, int inc);
void main (){
  int i, datos[]={19,20,36,45,54,69,70,88,97,101};

  incrementar(datos, 10, 5);

   for (i=0; i<10; i++)
         printf("%d\n", datos[i]);

}

void incrementar(int vector[], int N, int inc){
  int i;
   for (i=0; i< N; i++)
       vector[i] += inc ;
}
```

Ordenamiento - función qsort ()

void **qsort(**void ***base,**
size_t **num,**
size_t **tam,**
int **(*compara)(const void * a, const void* b))**

parámetros

- **base** - un puntero al primer elemento del arreglo a ordenar.
- **num** - el número los elementos del arreglo.
- **tam** - el tamaño de cada elemento del arreglo, en bytes.
- **compara** - Función utilizada para comparar dos elementos.

Retorno	Descripción
<0	El elemento ***a*** va antes del elemento apuntado por ***b (Ascendente)***
0	El elemento ***a*** es equivalente al elemento ***b (Sin cambio)***
>0	El elemento ***a*** va después del elemento ***b (Descendente)***

Ordenamiento Ascendente

```
#include <stdio.h>
#include <stdlib.h>
int asc (const void * a, const void * b){
   return ( *(int*)a - *(int*)b );
}
int main(){
   int N, i;
   printf("Numero de datos a ordenar:");
   scanf("%d",&N);
   int datos[N];
   for( i = 0 ; i < N; i++ ) {
      scanf("%d", &datos[i]);
   }
   qsort(datos, N, sizeof(int), asc); // Función de ordenación
   printf("\nDatos Ordenados:\n");
   for( i = 0 ; i < N; i++ ) {
      printf.("%d ", datos[i]);
   }
  return 0;
}
```

Ordenamiento Descendente

```
#include <stdio.h>
#include <stdlib.h>
int desc (const void * a, const void * b){
   return ( *(int*)b - *(int*)a );
}
int main(){
   int N, i;
   printf("Numero de datos a ordenar:");
   scanf("%d",&N);
   int datos[N];
   for( i = 0 ; i < N; i++ ) {
      scanf("%d", &datos[i]);
   }
   qsort(datos, N, sizeof(int), desc); // Función de ordenación
   printf("\nDatos Ordenados:\n");
   for( i = 0 ; i < N; i++ ) {
      printf("%d ", datos[i]);
   }
  return 0;
}
```